FORMATO PARA ELABORACIÓN DE TALLERES ESTUDIANTILES	Departamento: Eléctrica y Electrónica
	Carrera: Electrónica y Automatización Asignatura: Fundamentos de la Programación NRC:28561
	Apellidos y nombres del estudiante: Figueroa Cerón Sofía Vanessa

Ejercicio 10

Ejercicio: Números primos comprendidos entre dos números dados

10.Desarrolle un programa que muestre por pantalla todos los números primos comprendidos entre dos números introducidos por teclado.

Resultado esperado	Leer dos números enteros ingresados por teclado y mostrar en pantalla todos los números primos comprendidos entre esos dos valores..

Contexto del ejercicio

La condición principal es recorrer cada número desde a hasta b y verificar si solamente es divisible para 1 y para sí mismo. Para esto, se usa una variable llamada esPrimo, que indica si el número evaluado cumple o no con la condición de primo.

Regla de cálculo:

```
Si i <= 1 Entonces
   esPrimo <- 0
FinSi
```

Luego se revisa si el número tiene divisores entre 2 e i - 1:

```
Si i MOD j = 0 Entonces
   esPrimo <- 0
FinSi
```

1. Seudocódigo para PSeInt

```
Inicio

   Definir a, b, i, j, esPrimo Como Entero



   Escribir "Ingrese el primer número:"

   Leer a
```

```
    Escribir "Ingrese el segundo número:"
    Leer b

    Para i <- a Hasta b Hacer
        esPrimo <- 1

        Si i <= 1 Entonces
            esPrimo <- 0
        FinSi

        Para j <- 2 Hasta i - 1 Hacer
            Si i MOD j = 0 Entonces
                esPrimo <- 0
            FinSi
        FinPara

        Si esPrimo = 1 Entonces
            Escribir i
        FinSi
    FinPara
Fin
```

2. Prueba de escritorio en PSeInt

Paso	i	jcorrido	esPrimo	Salida	Descripción
1					Se declaran las variables.
2					Se ingresan a = 2 y b = 10

3	2	No corre	1	2	El número 2 es primo.
4	3	2	1	3	El número 3 es primo.
5	4	2	0	.	Como 4 MOD 2 = 0, no es primo.
6	5	2,3,4	1	5	El número 5 es primo.
7	6	2	0		Como 6 MOD 2 = 0, no es primo.
8	7	2,3,4,5,6	1	7	El número 7 es primo.
9	8	2	0		Como 8 MOD 2 = 0, no es primo.
10	9	3	0		Como 9 MOD 3 = 0, no es primo.
11	10	2	0		Como 10 MOD 2 = 0, no es primo.

Salida esperada al ingresar:

Salida esperada al ingresar:

Ingrese un número positivo:15

La raíz cuadrada entera es: 3

3. Guía para pasar de PSeInt a Code::Blocks con sintaxis validada

Ahora sí: pasamos la lógica a lenguaje C. La clave es traducir cada instrucción de PSeInt a su equivalente en C, sin perder el orden de los ciclos.

Pseint	C / Code::Blocks	Para Recordar
Definir a, b, i, j, esPrimo Como Entero	int a, b, i, j, esPrimo;	Las variables enteras en C se declaran con int.
Escribir "Ingrese el primer número:"	printf("Ingrese el primer numero: ");	printf muestra mensajes en pantalla.
Leer a	scanf("%d", &a);	%d se usa para leer números enteros.
Escribir "Ingrese el segundo número:"	printf("Ingrese el segundo numero: ");	Se pide el límite final del interval
Leer b	scanf("%d", &b);	El & guarda el valor dentro de la variable.
Para i <- a Hasta b Hacer	for (i = a; i <= b; i++) {	El ciclo recorre todos los números desde a hasta b.
esPrimo <- 1	esPrimo = 1;	Se asume al inicio que el número sí es primo
esPrimo <- 0	esPrimo = 0;	El 0 representa que no es primo.
Para j <- 2 Hasta i - 1 Hacer	for (j = 2; j <= i - 1; j++) {	Se buscan divisores entre 2 y el número anterior.
Si i MOD j = 0 Entonces	if (i % j == 0) {	En C, % calcula el residuo.
Si esPrimo = 1 Entonces	if (esPrimo == 1) {	Si sigue siendo primo, se imprime.
Escribir i	printf("%d\n", i);	Se muestra el número primo encontrado.
FinSi / FinPara	}	Las llaves cierran condiciones y ciclos en C.

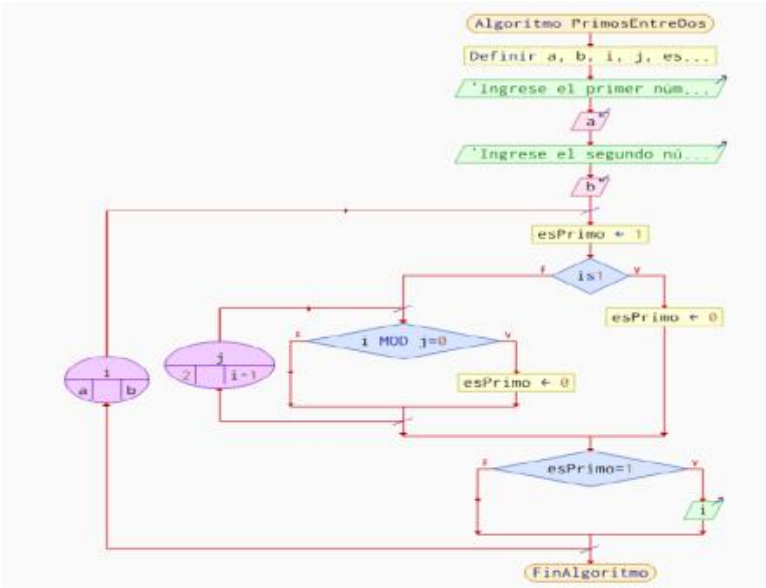
Código validado en C para Code::Blocks

```
#include <stdio.h>


int main() {

    int a, b, i, j, esPrimo;


    printf("Ingrese el primer numero: ");

    scanf("%d", &a);
```

```
    printf("Ingrese el segundo numero: ");
    scanf("%d", &b);

    for (i = a; i <= b; i++) {
        esPrimo = 1;

        if (i <= 1) {
            esPrimo = 0;
        }

        for (j = 2; j <= i - 1; j++) {
            if (i % j == 0) {
                esPrimo = 0;
            }
        }

        if (esPrimo == 1) {
            printf("%d\n", i);
        }
    }

    return 0;
}
```

Diagrama De Flujo

Ejercicio 10



Conclusiones

El programa permite mostrar correctamente los números primos comprendidos entre dos valores ingresados por teclado. Para lograrlo, recorre cada número del intervalo y verifica si tiene divisores diferentes de 1 y de sí mismo. Si no encuentra divisores, el número se considera primo y se muestra en pantalla.

Recomendaciones

Se recomienda verificar bien el uso del operador MOD en PSeInt y % en C, porque permite saber si un número es divisible para otro. También es importante recordar que los números menores o iguales a 1 no son primos, por eso se debe colocar esa condición dentro del programa.

Pasos sugeridos en Code::Blocks

1. Crear un nuevo archivo con extensión .c o abrir el archivo del ejercicio.
2. Copiar el código validado en C.
3. Compilar con Build o F9
4. Ejecutar con Run o con Build and Run.F9
5. Comparar la salida con la prueba de escritorio.

4. Rúbrica de evaluación sobre 20 puntos

Criterio	Excelente	Bueno	En proceso	Puntaje
Comprensión del problema	Identifica claramente que el triángulo debe ser hueco, con borde izquierdo, diagonal y base.	Reconoce la forma del triángulo, aunque explica parcialmente la condición de impresión.	Confunde triángulo lleno con triángulo hueco o no distingue los bordes.	4 pts
Seudocódigo en PSeInt	Usa correctamente variables, Repetir, Para, Si/Sino y Escribir Sin Saltar.	Presenta la estructura general, con pequeños errores de sintaxis o ubicación.	El seudocódigo está incompleto o no representa la lógica del ejercicio.	4 pts
Prueba de escritorio	Comprueba el algoritmo fila por fila y explica la salida para n = 10.	Realiza una prueba parcial con algunas filas o poca explicación.	No evidencia seguimiento de variables i y j o la salida no coincide.	4 pts
Traducción a C en Code::Blocks	El código compila, ejecuta y mantiene la lógica validada del triángulo hueco.	El código tiene errores menores corregibles, pero la lógica principal es adecuada.	El código no compila o no aplica correctamente FOR, if y operadores lógicos.	5 pts
Presentación y orden	Entrega el trabajo limpio, ordenado y fácil de leer, con comentarios útiles.	El trabajo es comprensible, aunque puede mejorar en orden o comentarios.	El trabajo es difícil de seguir o no evidencia organización.	3 pts